

生成臨床 UI の更新時に生じる情報の不一致を評価する TraceBench-EHR

TraceBench-EHR: Evaluating Cross-Layer Integrity in Generated Clinical Interfaces

平田 蓮

Ren Hirata

要旨 生成 AI による臨床 UI の更新では、SQL と画面が個別に動作しても、臨床質問、SQL、実行結果、表示の対応が壊れ得る。本研究は、この問題を検査する評価基盤 TraceBench-EHR を構築する。別患者の参照や期間条件の欠落など 8 種類の不整合を意図的に作り、個別確認から質問と表示の一括照合まで 4 種類の検査方法を比べる。最終評価前の動作確認では 931 件中 924 件から基準を生成し、758 組、6,064 候補を得た。各不整合は 34~133 種類の質問形式を含み、候補生成とは別の正解判定との不一致は 0 件だった。最終評価用データは未使用であり、現時点では評価手順の成立だけを確認済みとする。

1. はじめに

電子カルテには、患者属性、入退院、検査、処方、処置など、診療で参照される情報が複数の表と画面に分かれて蓄積される。医療従事者が必要とする情報は診療場面と確認目的によって変わるため、同じ患者データであっても、表示すべき項目、期間、集約方法、提示形式は一意ではない。タスクに応じて画面を生成・更新する臨床ユーザーインターフェース (UI) は、この情報探索を支える一つの方法である。

この更新では、個々の成果物が正常でも、成果物間の対応が壊れる場合がある。たとえば「対象患者の直近 24 時間の腎機能を数値で表示する」という要求に対し、SQL だけが別患者を参照する、24 時間の条件だけが外れる、SQL を更新した後も古い結果が残る、といった候補である。SQL は読み取り専用で実行でき、結果と表示部品も描画できるため、各成果物を個別に検査するだけでは拒否できない。それでも、要求した患者、期間、項目と画面に出る値の対応は壊れている。本稿では、この誤りを**層間不整合**と呼ぶ。

近年の生成 UI 研究では、自然言語から UI コードを直接作る方法に加え、タスクや意味を表す中間表現を介して UI を生成・編集する方法が提案されている。Jelly はタスク駆動データモデルを生成・更新し、そのモデルから UI を構成する [1]。Bridging Gulfs は設計意図の意味表現を、人の入力と生成 UI を結ぶ層として利用する [2]。SimStep は複数のグラフを用いて、生成結果の前提を人が段階的に確認・修正できるようにす

る [3]。これらの研究は検査可能な中間表現を提示するが、臨床情報要求からデータ取得、実行結果、表示までの不整合を、同じ候補と正解判定で比較する評価基盤は示していない。

本研究は、生成された臨床 UI の更新を検査する評価基盤を構築する。この評価基盤を TraceBench-EHR と呼ぶ。臨床質問と正解 SQL を収録した EHRSQL-2024[4] を、公開デモ用の診療データである MIMIC-IV Clinical Database Demo v2.2[5] 上で実行し、正しい基準表示を作る。その後、別患者の参照や期間条件の欠落など 8 種類の不整合を意図的に入れる。同じ更新候補を、SQL や表示部品を一つずつ確認する方法、各成果物の条件を確認する方法、質問から表示までを依存グラフでまとめて照合する方法、同じ対応内容を一枚の表で照合する方法へ入力する。後ろの二方式を比べることで、確認内容の効果とグラフ形式の効果に分ける。

本稿の貢献は次の三点である。第一に、臨床質問から表示までの対応が壊れる 8 種類の不整合を定める。第二に、不整合を同じ手順で繰り返し作り、候補生成とは別の処理で正解を判定する評価方法を示す。第三に、確認内容と保存形式を分けて比較し、検出だけでなく問題箇所の特定と一回の自動修正を評価する。臨床上の安全性、使いやすさ、認知負荷、臨床転帰は本評価の主張範囲に含めない。

2. 関連研究

2.1 生成・更新可能な UI

生成 UI では、利用者の要求から画面または実行可能なコードを作るだけでなく、生成後の変更をどう扱う

かが課題となる。直接生成されたコードを局所的に変更すると、変更対象以外の状態、データ参照、表示規則が意図せず変わる可能性がある。Jelly は、利用者の情報タスクをデータモデルとして保持し、自然言語と直接操作をモデルの変更へ変換する [1]。これにより、UI を作り直すたびにタスク構造が失われる問題へ対処する。

意味中間表現を用いる研究では、利用者が生成モデルの解釈を確認しやすくすることが重視される。Bridging Gulfs は、製品、機能、デザインシステム、部品などの意味を階層的に構造化し、入力意図と出力 UI の対応を提示する [2]。SimStep は Concept Graph、Scenario Graph、Learning Goal Graph、UI Graph を設け、人が各段階の仮定を修正する [3]。本研究も中間表現を使うが、グラフを紹介すること自体を利点とみなさず、情報要求、データ取得、実行結果、表示の対応を評価対象にする。

2.2 自然言語質問から SQL を作る研究

EHRSQL は、医療従事者から収集した情報要求を基に、電子カルテを対象とする自然言語質問と SQL の組を提供する。EHRSQL-2024 は MIMIC-IV Demo の表構造に対応し、回答可能な質問と回答不能な質問を含む共有タスクとして整備された [4]。自然言語から SQL を作る Text-to-SQL の評価では、質問に対して SQL を正しく生成できるか、回答不能な質問で棄権できるかが中心となる。

TraceBench-EHR は SQL 生成モデルの性能を比較しない。正解 SQL を基準として使用し、その実行結果を表示部品まで接続した後の更新を扱う。つまり、質問と SQL の一致に加えて、SQL と実行結果、実行結果と表示方式、更新後の各成果物が同じ内容を指しているかを検査する。EHRSQL を基盤にする理由は、質問形式、SQL、開発用データ、調整確認用データ、最終評価用データが公式に分けられており、最終結果を見る前に更新候補と評価規則を固定できるためである。以下では、公式の区分名が必要な箇所だけ train、validation、test を併記する。

2.3 モデル変換と整合性検査

構造化データとビューの更新を対応付ける問題は、双方向変換でも扱われてきた。Lens の研究は、元データとビューの間の往復変換が満たすべき性質を定式化している [6]。また、完全な正解出力を列挙しにくいモ

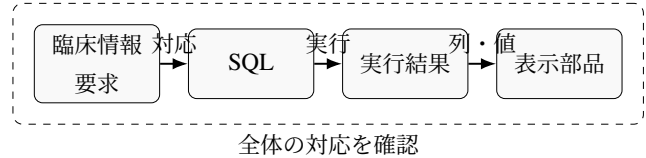


図1 TraceBench-EHR が検査する情報の流れ

デル変換に対して、不変関係を用いるメタモルフィックテストが提案されている [7]。本研究では完成した JSON の完全一致を求めず、変更すべき内容と保持すべき内容を述語として表す。

一般的なデータ形式の検査や画面描画の検査は必要であるが、これらは一つの成果物だけを確認する。質問から表示までの対応の誤りを見つけるには、複数の成果物間で同じ患者、項目、期間、集約、接続が維持されたかを確認する必要がある。そこで本研究は、個々の成果物だけを見る方法から、質問と表示をまとめて照合する方法までを同じ候補に適用する。

3. 研究質問と評価の対応

本研究では、増分更新において最終画面や個別成果物の検査を通過する不整合を対象とする。研究質問を次のように定める。

- RQ1：SQL や表示部品を一つずつ確認するだけでは見つからない不整合には、どのような種類があるか。
- RQ2：個別に確認する方法と、質問から表示までをまとめて照合する方法では、不整合の見逃し、正しい更新の受理、問題箇所の特定がどう異なるか。
- RQ3：検査で見つけた問題を一回だけ自動修正すると、どのような不整合を直せて、どのような不整合で失敗や不要な拒否が起きるか。

各研究質問を表1の対象と指標へ対応させる。RQ1は8種類の不整合ごとに、個別確認をすり抜けた候補数と具体例を調べる。RQ2は4種類の検査方法について、不整合を見逃した割合、正しい更新を受理した割合、問題箇所を正しく示した割合を比較する。RQ3は検査結果を使った一回の修正について、成功、誤った修正、拒否を分ける。

事前の予想は、成果物一つずつ確認するだけでは別患者の参照や期間条件の欠落を見逃し、質問から表示までをまとめて照合する方法は見逃しを減らしつつ正しい更新を受理するというものである。まとめて照合する二方式には同じ対応内容を持たせる。両者が同

表 1 研究質問と評価の対応

質問	比較対象	判断に使う結果
RQ1	8 種類の不整合	個別確認をすり抜けた数 と例
RQ2	4 種類の検査方法	見逃し、正しい受理、特 定
RQ3	一回の自動修正	成功、誤った修正、拒否

じ判定を返す場合、検出性能はグラフ形式ではなく確認内容によるものと解釈する。

4. TraceBench-EHR の設計

4.1 基準ケースの構築

入力は EHRSQL-2024 の回答可能な質問と正解 SQL である。回答不能ケースは、正解 SQL から基準 UI を構成できないため主評価から除外し、除外数を報告する。正解 SQL は読み取り専用の SQLite 接続で MIMIC-IV Demo v2.2 へ実行する。SQL ごとの実行上限は 5 秒とし、書き込みを含む文、複数文、許可しない構文は実行前に拒否する。

実行結果が単一値の場合は数値カード、複数行や複数列の場合は表を割り当てる。基準ケースは、質問形式、SQL、結果形状、データ取得、表示部品の対応を持つ。患者単位の値を対応条件へ埋め込まず、ケース ID、入力内容を識別するハッシュ値、結果形状、対応識別子を保存する。空結果は除外せず、列構造を持つ表として扱う。

同じ質問形式と SQL 構造を共有するケースを更新前後として組み合わせる。これにより、患者、項目、期間などの一部を変えた妥当な更新を作り、同じ基準から不整合候補を生成できる。同じ質問形式から作った複数の候補を独立した例として数えず、質問形式を分析単位とする。

4.2 8 種類の不整合

意図的に入れる不整合は表 2 の 8 種類である。各規則は一つの対応だけを壊し、それ以外の構造を保つ。複数の不整合が同じ候補へ偶然混入しないよう、生成後に別の処理で正解を判定する。

対象患者の不整合では、要求側の患者を保ったまま SQL の患者条件を別の候補へ変える。対象項目では、要求と異なる列または集約対象を取得する。期間では時間表現に対応する WHERE 条件や並び順を壊す。情

表 2 TraceBench-EHR の不整合分類

種類	壊す対応
対象患者	要求と SQL が参照する患者
対象項目	要求項目と取得列・集約対象
期間	時点・期間条件と SQL の絞り込み
情報源	必要情報と参照表
集約方法	件数、最大、最小、平均など
表示方式	結果形状と表示部品の種類
接続	データ取得と表示部品の参照
部分更新	SQL 更新後に残る古い結果

報源では同じような列名を持つ別表を参照させる。集約方法では、要求された最大値を件数へ変えるなど、SQL として実行可能なまま意味を変える。

表示方式の不整合では、SQL、実行結果、列、値を保ち、数値カードを表へ、データフレーム型の表を別の表部品へ変える。単独では描画できても、基準ケースが定めた結果形状と表示方式の対応が壊れる。接続の不整合では、表示部品が別のデータ取得を参照する。部分更新では SQL だけを新しい要求へ変更し、以前の実行結果を保持する。これにより、各成果物が個別に妥当でも、更新の同期が失敗する状況を作る。

4.3 独立した正解判定

候補生成と正解判定は別の処理にする。正解判定は、質問形式から得た患者、項目、期間、情報源、集約と、SQL の解析結果、実行結果の形状、データ取得と表示部品の参照を照合する。比較対象である実行時検査の判定を正解として使うと、同じ誤りを共有して評価値が高く見える可能性があるためである。候補の期待結果と別処理による正解判定が一致しない場合は、最終評価を停止する。

完全な UI JSON の一致を正解としない理由は、意味を保った複数の表現を許容するためである。基準と異なる安定 ID、表示文言、順序であっても、要求、データ、表示の意味条件を満たす候補は妥当とする。一方、見た目が同じでも別患者や古い結果を表示する候補は不整合とする。

4.4 候補と診断の例

基準ケースが「患者 A の直近 24 時間の検査 X を数値で表示する」である場合を考える。正しい更新として患者 B へ対象を変更するなら、質問側の患者、SQL の

表3 候補更新と期待する診断の例

候補	個別確認の結果	一括照合の結果
患者誤り	SQL 実行可能	対象患者
期間欠落	SQL 実行可能	期間
結果残留	表示可能	更新番号
参照誤り	描画可能	接続
表示変更	描画可能	表示方式

患者条件、実行結果の更新番号、データ取得と表示部品の参照を同じ更新として変更する必要がある。一つだけ古い状態に残れば、各成果物を単独では処理できても、更新全体として不整合になる。

対象患者の候補では、SQL 文自体が読み取り専用で、存在する患者を参照していれば個別確認を通過し得る。質問から表示までをまとめて照合すると、質問と SQL が指す患者の違いを対象患者の不整合として特定できる。期間の候補も同様に、時間条件を削除しても SQL 構文と実行可能性は保たれるため、要求の時間表現との照合が必要になる。

部分更新の候補では、更新後の SQL と更新前の実行結果がそれぞれ妥当な形式を持つ。そこで、SQL の内容を識別するハッシュ値と実行結果へ付与した更新番号を対応させ、古い結果が残ったことを診断する。接続の候補では、参照先のデータ取得が存在する限り表示できる場合があるため、情報要求から期待するデータ取得と表示部品の接続を検査する。

これらの例では、問題箇所を単に「検査失敗」と返さず、対象患者、期間、更新番号、接続、表示方式のいずれが壊れたかを記録する。この診断単位を修正内容と対応させることで、検出できた候補のうち、正しい値が一意に決まる不整合だけを一回の自動修正へ送る。

5. 検査システム

提案システムは、診療場面、情報要求、データ取得、表示部品をノードとする依存グラフを持つ。情報要求には確認または判断準備の内容を、データ取得には参照する EHR 情報と SQL を、表示部品には提示形式と配置を記録する。基準ケースでは、EHRSQL の一つの質問を一つの情報要求に対応させ、正解 SQL によるデータ取得と結果形状に対応する表示部品を接続する。

候補更新を受け取ると、検査器はデータ構造、読み取

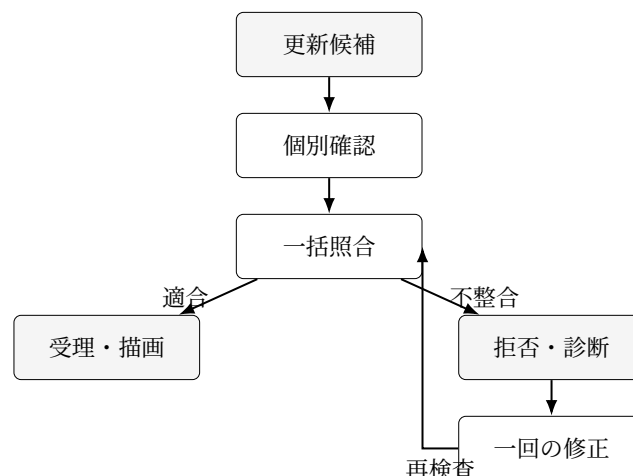


図2 候補の検査と一回の自動修正

り専用 SQL、SQL 実行、結果形状、描画可能性を確認する。質問から表示までをまとめて照合する条件では、さらに患者、項目、期間、情報源、集約、表示方式、接続、更新番号を確認する。不整合がある場合は、候補を画面へ反映せず、問題のある箇所と対応条件を返す。

自動修正は、この検査結果を一回だけ利用する。患者条件、期間条件、表示部品の参照など、正しい値が対応条件から一意に決まる場合は該当成果物を書き換える。意味が一意でない場合や、複数の成果物にまたがり安全な修正を決められない場合は拒否する。繰り返し試行して正解へ近づける方法は採らず、一回の修正について成功、失敗、誤った修正を区別する。

6. 実装と再現性

TraceBench-EHR の評価処理は、基準ケースの構築、候補生成、条件別検査、別処理による正解判定、統計集計に分ける。基準ケースの構築では、EHRSQL の質問と正解 SQL を読み込み、読み取り専用接続で SQL を実行する。候補生成では、同じ質問形式に属するケースを組み合わせ、固定した不整合を一つずつ入れる。ここまでの処理は乱数に依存せず、同じ入力と設定から同じ候補を得る。

条件別の検査では、生成された同一候補を 4 種類の方法へ適用する。各方法は候補を変更せず、確認する範囲だけを変える。検査結果には、受理・拒否、問題を特定した箇所、対応条件、自動修正の内容、検査時間を記録する。別処理による正解判定は候補生成と検査の途中結果を参照せず、保存された候補と正しい基準から期待結果を再計算する。これにより、候補を作っ

表 4 最終評価で保存する成果物

成果物	保存内容
候補の再生成情報	ケース ID、不整合の種類、ハッシュ値
条件別の判定	受理・拒否、問題箇所、自動修正の結果
集計結果	不整合別の件数、指標、95%信頼区間
実行記録	入力、設定、コード、出力の版

た規則の誤りが、そのまま正解判定へ流れ込むことを避ける。

最終評価は CPU だけで実行し、Gemini を含む外部モデルを呼ばない。乱数の初期値は 20260827、SQL ごとの上限は 5 秒、統計処理の再抽出回数は 10,000 回とする。実行時刻は計測値として保存するが、候補の同一性を表すハッシュ値には含めない。入力データ、設定、コード、候補、集計のハッシュ値を実行記録へ残り、別の出力先へ再実行した場合に候補と集計が一致するかを確認する。

保存する成果物を表 4 に示す。候補の再生成情報には候補を識別する最小情報を置き、条件別の判定には受理・拒否と診断を一実行一行で記録する。集計結果は質問形式単位で作る。実行記録はデータセット、コード、設定、実行環境、出力の版を結び付ける。内部のファイル名や関数名は再現条件ではないため、ここでは扱う情報と役割を示す。

テストでは、8 種類の不整合が意図した一つの対応だけを壊すこと、正しい候補が別処理による正解判定を満たすこと、依存グラフと一枚の対応表が同じ判定を返すこと、修正後に再検査が行われることを確認する。さらに、質問文、SQL、患者 ID、結果値を成果物へ書き出そうとした場合に停止する検査を設ける。機能テスト、静的なコード検査、型チェックがすべて成功しない限り評価版を固定しない。

再現性の対象は、許可された入力から候補、判定、集計を再生成できることまでである。MIMIC 由来の生データや患者単位の結果をリポジトリへ含めないため、第三者が再実行する場合は、各データセットの利用条件に従って入力を取得する必要がある。公開可能な成果物と再配布できない入力を分け、どの版を使ったかをハッシュ値で検証できる構成にする。

表 5 比較する検査条件

条件	検査範囲
個別確認	データ構造、SQL 安全性、実行、描画
成果物ごとの確認 依存グラフによる 照合	各成果物が持つ患者・項目など 質問から表示部品までの対応
一枚の表による照 合	グラフと同じ対応を表形式で保 持

7. 評価方法

7.1 比較条件

同じ基準ケースと候補を表 5 の 4 条件へ適用する。条件間で候補生成、SQL 実行環境、一般的なデータ構造の検査をそろえる。

個別確認は、SQL や表示部品が単独で実行可能かを調べる。成果物ごとの確認は、SQL、実行結果、表示部品に必要な患者や項目を個別に調べるが、成果物をまたぐ対応までは扱わない。依存グラフによる照合は、質問、データ取得、SQL、実行結果、表示部品の関係を線で結んで確認する。一枚の表による照合は、同じ対応内容を表形式で確認する。

依存グラフと一枚の対応表を比較に含めることで、確認する内容の効果と保存形式の効果を分ける。両条件の性能が一致した場合、グラフの一般的な優位性は主張しない。グラフは影響範囲の可視化や編集単位として別の利点を持ち得るが、それは本技術評価だけでは確定しない。

7.2 評価指標

主要評価は**不整合の見逃し率**と**正しい更新の受理率**である。不整合の見逃し率は、不整合を含む候補のうち受理されて UI へ反映可能になった割合であり、低いほどよい。正しい更新の受理率は、正しい候補のうち SQL 実行と描画まで完了して受理された割合であり、高いほどよい。二つを一つの正解率にまとめないのは、正しい更新まで拒否することで見逃しだけを減らす条件を区別するためである。

副次評価は、問題箇所の特定率、一回の自動修正の成功率、誤修正率、正しい候補の不要な拒否率、検査時間である。問題箇所は、不整合がある成果物と対応条件の組で採点する。修正成功は、修正後の候補が別

処理による正解判定を満たし、SQL 実行と描画まで完了することと定義する。拒否すべきケースを無理に修正した場合は誤修正として分ける。

条件差と 95% 信頼区間は、質問形式を単位に繰り返し再抽出する統計手法を 10,000 回適用して求める。この手法をクラスターブートストラップと呼ぶ。同じ質問から作った複数の不整合を独立した例として扱わない。平均だけでなく、不整合の種類別の分布と代表的な失敗例を報告する。

7.3 データの使い分けと事前基準

EHRSQL-2024 の開発用データ (train) で実装とテストを作り、調整確認用データ (validation) の回答可能 931 件で最終評価前の動作を確認する。確認後、不整合を入れる規則、別処理による正解判定、比較条件、分析コードを Git の記録で固定する。その後、最終評価用データ (test) の回答可能 934 件を一度だけ実行する。最終評価の開始後に評価規則の変更が必要になった場合は実行を停止し、その結果を正式な評価結果として使用しない。

最終評価へ進んでよいかを判断する事前基準は、入力のハッシュ値が一致すること、機能テスト・静的なコード検査・型チェックが成功すること、基準 SQL の実行とグラフ検査の成功率が 95% 以上であること、各不整合が 10 種類以上の質問形式と 30 件以上の候補を持つこと、保存成果物に質問文・SQL・患者 ID・結果値が混入していないことである。基準を先に定めることで、評価結果を見た後の規則変更を抑える。

8. 確認済みの結果

8.1 50 件を使った最初の動作確認

2026 年 8 月 27 日に、開発用データから異なる質問形式を優先して 50 件を選び、正解 SQL の実行と最小の依存グラフへの変換を確認した。結果を表 6 に示す。

全 50 件で正解 SQL を実行でき、依存グラフの検証に成功した。49 件で非空結果を取得し、1 件は列を持つ空結果だった。結果形状に基づき 40 件を数値カード、10 件を表へ割り当てた。タイムアウト、SQL エラー、書き込み拒否、グラフ検証エラーはなかった。この結果は、選定した 50 種類の質問を現行実装へ機械的に変換できることを示す。全ケースへの適用可能性や表示方式の臨床的妥当性を示すものではない。

初回の成果物では、SQLite が返す列名に SQL 断片が

表 6 EHRSQL 50 件を使った最初の動作確認

確認項目	結果
正解 SQL の実行成功	50/50
非空結果	49/50
依存グラフの検証成功	50/50
数値カードへの割当	40/50
表への割当	10/50
タイムアウト・書込拒否・SQL エラー	0

含まれるケースがあった。この出力を無効とし、列名を成果物へ保存しないよう修正して再実行した。最終成果物には質問文、SQL、患者 ID、患者単位の結果値が含まれないことを確認した。この事例から、評価指標だけでなく、生成物への機微情報の混入を最終評価に進めない条件として機械的に検査する必要が分かった。

8.2 小規模な事前確認

TraceBench-EHR の設計前に、更新方法の比較が実行できるかを小規模に確認した。2026 年 7 月 22 日に、合成した 24 ケースと 120 件の共通変更仕様を用い、同じ更新要求と安全要件を持つ直接差分方式と完全グラフ方式を比較した。各ケースは正しい候補 1 件、単一の不整合 3 件、複数の不整合を含む候補 1 件を持つ。比較中に生成モデルを呼ばず、候補は同じ入力から同じ結果になるよう作成した。実験ファイル上の版番号は v0.4 である。

直接差分方式と完全グラフ方式は、ともに不整合を含む候補 96 件をすべて拒否し、正しい候補 24 件をすべて受理した。不整合の見逃し率と正しい更新の受理率について、二方式の差は 0.0 で、95% 信頼区間も 0.0 から 0.0 だった。グラフ方式から変更対象外の保護、安全条件、対応関係の確認を個別に外すと、それぞれに対応する不整合 18 件が通過した。

この結果は、同じ確認内容を実装した範囲で、グラフ形式そのものが直接差分より高い検出性能を持つという予想を支持しなかった。一方、各確認を外したときに対応する不整合が通過したため、更新範囲、安全条件、対応関係を明示的に確認する役割は示された。そこで最終評価では、グラフ方式に弱い基準だけを置かず、同じ対応内容を持つ一枚の表を比較条件にする。

表7 修正版で行った最終評価前の確認

確認項目	結果
回答可能ケース	931
基準生成成功	924 (99.25%)
更新前後の組	758
4条件へ入力した候補	6,064
不整合ごとの質問形式	34~133
正解判定との不一致	0
グラフと対応表の不一致	0
機微情報を含む保存成果物	0

8.3 最終評価前の動作確認

最初の評価設計では、調整確認用データ 1,163 件のうち正解 SQL を持つ 931 件を対象とした。SQL 実行は 924 件で成功し、成功率は 99.25% だった。625 組、5,000 候補を評価したが、当初予定した列順の不整合は候補 0 件、質問形式 0 件となった。実行に成功した SQL 結果がすべて 1 列であり、列順を変えて意味的な対応を壊す候補を作らなかったためである。この最初の評価設計を実験記録では v1.0 と呼ぶ。

列順の不整合が事前の最小件数を満たさなかったため、最初の設計は固定しなかった。列順を表示方式の不整合へ置き換え、修正版を作った。他の 7 種類の不整合、4 種類の検査方法、評価指標、分析方法、乱数の初期値、データ管理、最終評価へ進むための基準は維持した。この修正版を実験記録では v1.1 と呼ぶ。

修正版の再実行では、924 件の基準ケースから 758 組、6,064 候補を生成した。表 7 に示すとおり、8 種類の不整合はそれぞれ 34~133 種類の質問形式を含み、各不整合が 10 種類以上の質問形式と 30 件以上の候補を持つという事前基準を満たした。候補生成とは別の正解判定との不一致は 0 件、依存グラフと一枚の対応表による判定の不一致も 0 件だった。保存成果物に質問文、SQL、患者 ID、患者単位の結果値が含まれないことを確認した。

この結果から判断できるのは、候補生成、正解判定、検査方法の対応、機微情報を保存しない条件が最終評価前の基準を満たしたことだけである。最終評価用データはまだ実行していないため、4 種類の検査方法について、不整合の見逃し率、正しい更新の受理率、問題箇所の特定制率、自動修正の成功率は未確定である。

9. 考察

9.1 RQ1 個別確認では見つからない不整合

50 件を使った最初の動作確認では、正解 SQL を依存グラフと表示部品へ変換できた。一方、これは情報を正しく接続した基準ケースの確認であり、更新後に対応が保たれることを保証しない。SQL が実行できること、結果が表であること、表示部品が描画できることは、すべて必要条件である。しかし、患者、項目、期間、集約、表示の意味が一致しているかは別の検査を要する。

成果物間の不整合は、最終画面の見た目から判別しにくい。別患者の値でも数値カードとして表示でき、古い結果でも描画エラーは起きない。したがって、画面の画像比較や UI コードの構文検査だけでは、情報要求に対する正しさを判定できない。8 種類のうち、どれが個別確認を何件通過するかは最終評価用データで確定する。

9.2 RQ2 確認する内容とグラフ形式

小規模な事前確認で直接差分方式と完全グラフ方式に差がなかったことは、グラフを用いれば自動的に整合性が得られるという説明を支持しない。同じ確認内容を成果物側に実装できるなら、検出性能は同じになる可能性がある。そこで TraceBench-EHR は、グラフと同じ情報を持つ一枚の対応表を比較する。

修正版を使った最終評価前の動作確認では、依存グラフと一枚の対応表による判定の不一致が 0 件であることを確認した。これは二つの方法へ同じ確認内容を実装できたことを示すだけで、性能差がないという結果ではない。不整合の見逃し率や受理率の差は最終評価用データで判断する。本稿では、JSON、関係表、グラフデータベースという保存形式の一般的な優劣を主張しない。

9.3 RQ3 一回の自動修正で扱える範囲

対応条件から正しい値が一意に決まる不整合は、自動修正の対象にできる。たとえば表示部品が参照すべきデータ取得、SQL 更新後に対応する結果の更新番号、基準ケースで定めた表示方式は、検査結果から修正内容を決められる。一方、自然言語要求が複数の解釈を許す場合や、表示の細かさと臨床的な意味の選択が必要な場合、技術的な対応条件だけで正解を決められない。

そのため、自動修正の成功率を高くすることだけを

目的にしない。誤った修正と正しい候補の不要な拒否を分け、安全に正解を一つに決められない場合は拒否する。修正成功率、誤修正率、拒否率は最終評価用データを実行して初めて確定する。将来、人による評価を行う場合は、成果物の差分だけを提示する条件と、影響範囲、根拠、保護条件を提示する条件を比較し、専門家が誤りを検出・修正できるかを調べる。

9.4 評価方法を作る過程で分かったこと

最初の動作確認で列順の不整合を作れなかったことから、想定する不整合を概念だけで固定せず、実データの結果形状で生成件数を確認する必要が分かった。列順を表示方式へ変更した修正版は、8種類すべてで事前の最小件数を満たした。変更理由、維持した条件、再実行結果を版ごとに残すことで、最終評価の結果を見た後の規則変更と区別できる。

また、質問形式を単位に集計する必要がある。同じ質問形式から患者や項目だけを変えたケースは、SQL構造と不整合を入れる規則を共有する。候補を一件ずつ独立した例として数えると、特定の質問形式が結果へ過大に寄与する。質問形式を単位に繰り返し再抽出する統計処理と、不整合の種類別の報告により、この偏りを抑える。

10. 制約と倫理

本研究は MIMIC-IV Clinical Database Demo v2.2 を用いる技術評価である。評価から、臨床上の安全性、診療判断の正確さ、使いやすさ、認知負荷、臨床転帰、実運用、他施設への一般化を主張しない。正解 SQL は技術的な基準であり、表示内容の医学的な妥当性を専門家が確認した基準ではない。

人を対象とする評価は初回の主評価から外している。既存の専門家評価案は、倫理審査と実施許可の範囲、募集方法、データ保存、同意、撤回方法が確認されるまで実施しない。操作手順から所要時間を見積もる KLM という方法で行った予備確認も、読解、見落とし、判断、認知負荷を示す結果として扱わない。

生データ、質問文、正解 SQL、患者 ID、患者単位の結果値は GitHub と Notion へ保存しない。Git 管理外の一時領域だけで扱い、公開成果物にはケース ID、入力と候補のハッシュ値、不整合の種類、判定結果、集計、実行記録を保存する。MIMIC 由来の派生成果物に関する利用条件を確認し、データセットのライセンス

表 8 今後の作業

時期	作業
9月5日まで	評価版の確認と Git の記録による固定
9月6日	最終評価用 934 件の一回実行と別処理での集計
9月6～8日	指標、95% 区間、失敗例、自動修正例の確定
9月9日 修士論文	原稿、匿名化、再現資料の最終確認 技術評価と将来の人対象評価を分けて統合

と出典を明記する。

11. 次の作業と完了条件

修士論文までの作業を表 8 に示す。修正版を使った最終評価前の動作確認は 2026 年 8 月 27 日に完了した。次は評価規則、設定、分析コード、入力のハッシュ値、機微情報を保存しない条件を確認し、9月5日までに評価版を Git の記録で固定する。完了条件は、機能テスト、静的なコード検査、型チェックが成功し、保存成果物に質問文、SQL、患者 ID、患者単位の結果値がなく、別の集計処理で候補数と事前基準を再現できることである。

固定後、最終評価用データの回答可能 934 件を一度だけ実行する。入力のハッシュ値の不一致、テストの失敗、基準生成成功率 95% 未満、不整合の最小件数不足、機微情報の保存を検出した場合は停止し、その実行を正式結果に使わない。通過した場合は、不整合の種類ごとの見逃し率、受理率、特定率、自動修正の成功率と 95% 信頼区間を 9月8日までに確定する。評価規則の変更が必要な場合は版を上げ、最終評価の結果を見た後の変更として記録する。

修士論文では、試作システムの設計、小規模な事前確認、TraceBench-EHR の設計と最終評価を一つの流れとして整理する。人を対象とする評価は、技術評価で検出・特定・修正できる範囲を確定した後の課題として分ける。臨床 UI の利用効果を扱う場合は、専門家確認済みの症例と採点基準を用い、タスク成功率、成功した試行の完了時間、操作数、作業負荷を測る NASA-TLX、使いやすさを測る SUS を別々に報告する。

12. まとめ

本研究は、生成臨床 UI の更新で、臨床情報要求、SQL、実行結果、表示部品の対応が壊れる問題を扱う。TraceBench-EHR は EHRSQL-2024 と MIMIC-IV Demo を基に正しい表示の基準を作り、8 種類の不整合を同じ手順で入れる。個別確認、成果物ごとの確認、依存グラフによる照合、同じ内容を持つ一枚の表による照合を同じ候補で比較し、不整合の見逃し、正しい更新の受理、問題箇所の特典、一回の自動修正を評価する。

現在までに、異なる 50 種類の質問で正解 SQL の実行と依存グラフへの変換を確認した。小規模な事前確認では、同じ確認内容を持つ直接差分方式とグラフ方式の検出性能に差がなかった。修正版を使った最終評価前の動作確認では、924 件の基準ケースから 758 組、6,064 候補を生成し、8 種類の不整合が事前基準を満たした。候補生成とは別の正解判定との不一致は 0 件だった。最終評価用データは未実行であるため、現時点では評価手順の成立だけを確認済みとする。評価版を固定した後に最終評価用データを一度だけ実行し、3 つの研究質問へ回答する。

参考文献

- [1] Yining Cao, Peiling Jiang, and Haijun Xia. Generative and malleable user interfaces with generative and evolving task-driven data model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–20, 2025.
- [2] Seokhyeon Park, Soohyun Lee, Eugene Choi, Hyunwoo Kim, Minkyu Kweon, Yumin Song, and Jinwook Seo. Bridging gulfs in UI generation through semantic guidance. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pp. 1–16, 2026.
- [3] Zoe Kaputa, Anika Rajaram, Vryan Feliciano, Zhuoyue Lyu, Maneesh Agrawala, and Hariharan Subramonyam. SimStep: Human-in-the-loop authoring of interactive educational simulations through task-level abstractions. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pp. 1–34, 2026.
- [4] Gyubok Lee, Sunjun Kweon, Seongsu Bae, and Edward Choi. Overview of the EHRSQL 2024 shared task on reliable text-to-SQL modeling on electronic health records. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*, pp. 644–654. Association for Computational Linguistics, 2024.
- [5] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Steven Horng, Leo Anthony Celi, and Roger Mark. MIMIC-IV. *PhysioNet*, 2023. Version 2.2.
- [6] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, Vol. 29, No. 3, 2007.
- [7] Xiao He, Zhenjiang Li, Zhenjiang Hu, Xiaoyu Liu, and Takashi Hirayama. Testing bidirectional model transformation using metamorphic testing. *Information and Software Technology*, Vol. 104, pp. 48–69, 2018.